

Custom Boards

Zephyr RTOS Course





Objectives

- **Custom board** creation
- Understanding **Kconfig's** role
- Board **integration**
- **Custom application** build example





Quick Recap

- Kconfig vs. DTS file.
- Kconfig variables are software switches.
- The DTS file is the hardware map.





Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



Board Definition

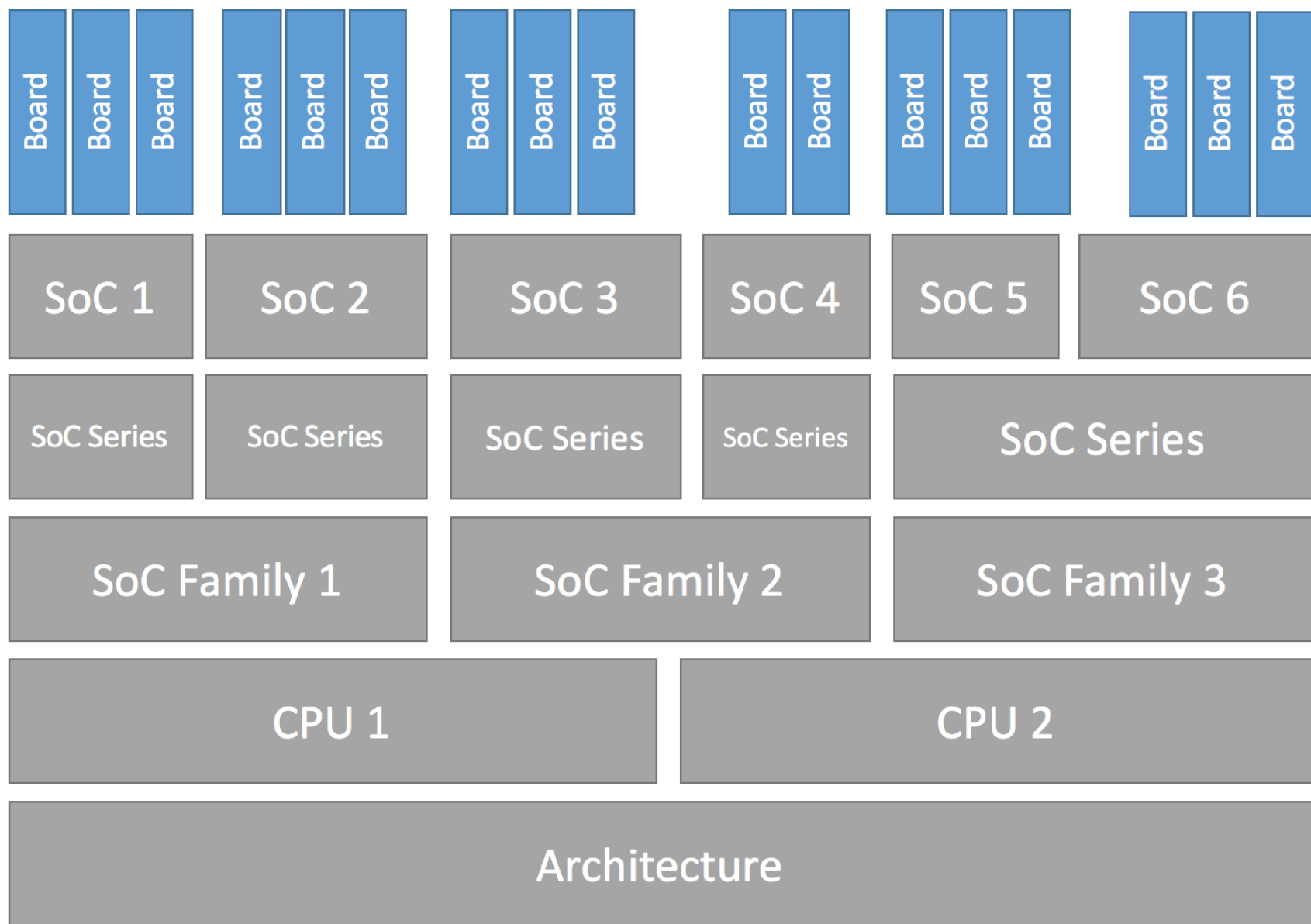


What Is a Board?



A **board** in Zephyr represents a complete hardware platform:

- **More than just a PCB** — it's a hardware abstraction layer
- Includes the **SoC** (microcontroller), **peripherals**, and **pin mappings**
- Contains all **configuration** needed to run Zephyr on that specific hardware
- Defined through **DTS** (Device Tree Source) and **Kconfig** files





Board Structure & Naming



```
board_name@revision/soc/cpu_cluster/variant
```

Example: `nrf5340dk/nrf5340/cpuapp` is a board for the nRF5340 DK

- **Board name:** Unique identifier (e.g., `nrf5340dk`)
- **Board revision:** Hardware version (e.g., `@1.2.0`)
- **SoC:** System-on-Chip (e.g., `nrf5340`)
- **CPU cluster:** Specific CPU core (e.g., `cpuapp` - application core)
- **Variant:** Build variant (e.g., `ns` - non-secure)



Creating Custom Boards



Two Approaches

- **Copy and Rename** — Start with existing board and modify
- **From Scratch** — Build complete board definition manually





Common to Both Methods



Board's directory layout, minimal files

```
└─ boards/  
  └─ our_board/  
    ├── board.yml  
    ├── our_board.dts  
    └─ Kconfig.our_board
```

Good to have

```
└─ board.cmake  
└─ Kconfig.defconfig  
└─ our_board_defconfig  
└─ our_board_pinctrl.dtsi
```



The Copy and Rename Method



1. **Choose board** to copy with the same MCU or SoC we want to use. Consult [\(1\)](#).
For example the nucleo_f429zi.
2. **Copy** all of its contents to **our_board** directory
3. **Rename** these files

```
nucleo_f429zi.yaml      →  our_board.yaml  
Kconfig.nucleo_f429zi  →  Kconfig.our_board  
nucleo_f429zi_defconfig →  our_board_defconfig  
nucleo_f429zi.dts      →  our_board.dts
```



Edit the Files - .yaml



`board.yaml` describes the board for West build.

```
board:
  name: board-name
  vendor: board-vendor
  revision:
    format: major.minor.patch|letter|number|custom
    default: default-revision-value
    exact: true|false
    revisions:
      - name: revA
      - name: revB
  socs: {...}
```



```
board:
  name: board-name
  vendor: board-vendor
  revision: {...}
  socs:
    - name: soc-1
      variants:
        - name: variant-1
        - name: variant-2
          variants:
            - name: sub-variant-2-1
    - name: soc-2
```



- **Kconfig.our_board** – This file creates the boards Kconfig and makes it available for the build system to select and use.

```
config BOARD_NUCLE0_F429ZI
    select SOC_STM32F429XX
```

- **Kconfig.defconfig** – Board-specific default values for Kconfig options.

```
if BOARD_NUCLE0_F429ZI
if NETWORKING
config NET_L2_ETHERNET
    default y
endif # NETWORKING
endif # BOARD_NUCLE0_F429ZI
```



Edit the Files – Result



```
board.cmake  
board.yml  
Kconfig.defconfig  
Kconfig.nucleo_f429zi  
nucleo_f429zi_defconfig  
nucleo_f429zi.dts  
nucleo_f429zi.yaml
```



```
board.cmake  
board.yml  
Kconfig.defconfig  
Kconfig.our_board  
our_board_defconfig  
our_board.dts  
our_board.yaml
```



Change the Hardware



Now that we have our copied board, we can make our desired modifications. For example we can change the pin assignment for led_1.

```
/* our_board.dts */
leds: leds {
    compatible = "gpio-leds";
    green_led_1: led_1 {
        gpios = <&gpio1 1 GPIO_ACTIVE_HIGH>; /* Changed GPIO assigned to led */
        label = "User LD1";
    };
};
```



Build for our New Board



The following command builds the application for our new board

```
west build -b our_board --board-dir boards/our_board
```



Build for our New Board



Not to specify each time the board's directory, we can add the following line to our project's `CMakeLists.txt`, right before the `find_package()` line.

```
# In our project's main CMakeLists.txt
list(APPEND BOARD_ROOT ${CMAKE_CURRENT_SOURCE_DIR})
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
```

Now we can run

```
west build -b our_board
```



Our Board From Scratch



Minimal repository layout.

```
|— boards/
|   |— our_board/
|       |— board.yml <-- Metadata
|       |— our_board.dts <-- The hardware definition
|       |— Kconfig.our_board <-- Tells Zephyr our_board exists
```



Board Files Architecture



Build System Identification

board.yml

Kconfig.our_board

Register board
with build system

Board Configuration

Kconfig.defconfig

our_board_defconfig

Software features
& default settings

Hardware Mapping

our_board.dts

↓ includes

SoC DTSI
(stm32f429Xi.dtsi)

our_board_common.dtsi

our_board-pinctrl.dtsi

Device tree &
pin assignments



Entry Point: `our_board.dts` File



```
/dts-v1/;
#include <st/f4/stm32f429Xi.dtsi>
#include <st/f4/stm32f429zitx-pinctrl.dtsi>
#include <zephyr/dt-bindings/input/input-event-codes.h>
/ {
    model = "Our Board";
    compatible = "iomico,our-board";
    chosen {
        zephyr,flash = &flash0;
        zephyr,sram = &sram0; /* Point to regular RAM */
        zephyr,code-partition = &slot0_partition; /* Regular application slot */
    };
};
```



```
#include <zephyr/dt-bindings/pinctrl/stm32-pinctrl.h>
/ {
    soc {
        pinctrl: pin-controller@40020000 {
            /omit-if-no-ref/ adc1_in0_pa0: adc1_in0_pa0 {
                pinmux = <STM32_PINMUX('A', 0, ANALOG)>;
            };
            /omit-if-no-ref/ adc1_in1_pa1: adc1_in1_pa1 {
                pinmux = <STM32_PINMUX('A', 1, ANALOG)>;
            };
            ...
        };
    };
}
```



Kconfig



Kconfig.our_board

Just like in the previous method of creating our custom board, here we let West know the existence of our board.

```
config BOARD_NUCLE0_F429ZI
    select SOC_STM32F429XX
```



Kconfig.defconfig

```
if BOARD_NUCLE0_F429ZI

if NETWORKING
config NET_L2_ETHERNET
    default y
endif # NETWORKING

endif # BOARD_NUCLE0_F429ZI
...
```



our_board_defconfig



```
# Enable MPU
CONFIG_ARM_MPU=y
# Enable HW stack protection
CONFIG_HW_STACK_PROTECTION=y
CONFIG_SERIAL=y
# console
CONFIG_CONSOLE=y
CONFIG_UART_CONSOLE=y
# Enable GPIO
CONFIG_GPIO=y
```



Optional CMakeLists.txt – C Source Code



■ CMakeLists.txt

```
zephyr_library()  
zephyr_library_sources(board.c)
```

■ board.c

```
#include <zephyr/init.h>  
#include <zephyr/kernel.h>  
static int board_our_board_init(void) {  
    /* Some function to run before boot */  
}
```



Optional board.cmake



```
board_runner_args(stm32cubeprogrammer "--port=swd" "--reset-mode=hw")
board_runner_args(jlink "--device=STM32F429ZI" "--speed=4000")

include(${ZEPHYR_BASE}/boards/common/stm32cubeprogrammer.board.cmake)
include(${ZEPHYR_BASE}/boards/common/openocd-stm32.board.cmake)
include(${ZEPHYR_BASE}/boards/common/jlink.board.cmake)
```



Build and Flash



Just like in the previous copy method:

```
west build -b our_board  
west flash
```



Homework



Home Task



Task 1

- Create a custom board using the "Copy/Rename" method.
- Build the hello world sample for said board.
- Place the board directory in `<project_root>/boards/`.
- Push and tag it as `l5-task1`.



Task 2

- Create a custom board using the "From Scratch" method.
- Build the hello world sample for said board.
- It must also print a message ("Board Initialized") before entering the main entry point of the application.
- Push and tag it as `l5-task2`.

Thank You

Questions?





Resources

1. [Zephyr Supported Boards](#)
2. [Zephyr Board Porting Guide](#)
3. [Device Tree Documentation](#)

